

Fwolgen Wars - Guia Completo de Configuração do Ambiente On-Chain

Passo a passo completo, desde a ativação inicial até à compilação do contrato inteligente utilizando WSL (Ubuntu), Rust, Solana e Anchor.

1. Stack Tecnológica e Versões Oficiais

A tabela abaixo especifica as ferramentas e versões oficiais utilizadas para configurar o ambiente de desenvolvimento:

Ferramenta / Tecnologia	Versão / Especificação
Ambiente Base	Windows 11 com WSL2 (Distribuição Ubuntu)
Linguagem de Programação	Rust (via rustup / cargo)
Solana CLI	v1.18.26
Anchor CLI / AVM	Latest (gerenciado via Anchor Version Manager)

2. Instalação do Ambiente no WSL (Ubuntu)

Passo 2.1 — Instalar Dependências Básicas do Sistema

Abra o terminal do Ubuntu e execute os seguintes comandos:

```
sudo apt update && sudo apt upgrade -y  
sudo apt install build-essential pkg-config libssl-dev git curl wget -y
```

Passo 2.2 — Instalar o Compilador Rust (rustup / cargo)

Instale o ambiente Rust utilizando o instalador oficial:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh  
source "$HOME/.cargo/env"
```

Para verificar a instalação, execute:

```
rustc --version  
cargo --version
```

Passo 2.3 — Instalar a Solana CLI (v1.18.26)

Faça o download do pacote binário da Solana, extraia e configure o PATH:

```
cd ~
wget
https://github.com/solana-labs/solana/releases/download/v1.18.26/solana-release-x86_64-unknown-linux-gnu.tar.bz2
tar jxf solana-release-x86_64-unknown-linux-gnu.tar.bz2
echo 'export PATH="$HOME/solana-release/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Verifique a versão instalada:

```
solana --version
```

Passo 2.4 — Instalar o AVM e a Anchor CLI

Compile o Anchor Version Manager utilizando Cargo e instale a versão mais recente do Anchor:

```
cargo install --git https://github.com/coral-xyz/anchor avm --locked
avm install latest
avm use latest
```

Verifique a versão:

```
anchor --version
```

3. Configuração da Solana Devnet

Passo 3.1 — Configurar a Rede Devnet

Aponte a Solana CLI para a Devnet e confirme as configurações:

```
solana config set --url devnet
solana config get
```

Verifique principalmente: RPC URL, Cluster e Wallet Keypair.

Passo 3.2 — Verificar o Endereço da Carteira

Exiba o endereço da carteira local:

```
solana address
```

Se ainda não tiver uma carteira, crie uma nova:

```
solana-keygen new --no-bip39-passphrase
```

Passo 3.3 — Solicitar SOL de Teste via Airdrop

Solicite 2 SOL diretamente para a sua carteira:

```
solana airdrop 2
# Ou especifique explicitamente o endereço:
solana airdrop 2 $(solana address)
```

Passo 3.4 — Confirmar o Saldo

Verifique o saldo atualizado:

```
solana balance
```

Passo 3.5 — Solução de Problemas com Airdrop

A Devnet da Solana frequentemente aplica limites de taxa (rate limiting) ou passa por congestionamentos. Se o comando `solana airdrop 2` falhar com mensagens como HTTP 429 ou Rate limit reached, siga uma das alternativas:

- **Solicitar valores menores:** Tente solicitar apenas 1 SOL por vez (`solana airdrop 1`).
- **Usar o Faucet Web Oficial:** Acesse <https://faucet.solana.com>, cole o endereço obtido com `solana address`, selecione Devnet e clique para enviar.
- **Aguardar alguns minutos:** Tente novamente mais tarde caso o cluster esteja temporariamente sobrecarregado.

4. Inicialização do Projeto FOWLGEN WARS

Passo 4.1 — Criar o Projeto Anchor

Execute os comandos para inicializar e compilar o projeto base:

```
anchor init fowlgen_wars_contract
cd fowlgen_wars_contract
anchor build
```

5. Estrutura Principal do Projeto

Após a inicialização, a estrutura principal do projeto será semelhante à seguinte:

```
fowlgen_wars_contract/
├── Anchor.toml
├── programs/
│   └── fowlgen_wars_contract/
│       ├── src/
│       │   └── lib.rs
└── tests/
```

```
└─ target/
   Cargo.toml
```

Arquivos Principais: Descrição dos arquivos principais:

- **programs/fowlgen_wars_contract/src/lib.rs:** Contém a lógica principal do contrato inteligente em Rust.
- **Anchor.toml:** Contém as configurações do projeto, redes, carteira e scripts.
- **tests/:** Contém os scripts de teste do projeto.

6. Executar os Testes Locais

Passo 6.1 — Executar anchor test

Na raiz do projeto, execute:

```
cd ~/fowlgen_wars_contract
anchor test
```

O Anchor executará automaticamente: compilação do contrato, inicialização do validador local, deploy do programa na Localnet, execução dos testes e encerramento do validador. Caso o validador já esteja em execução, utilize:

```
anchor test --skip-local-validator
```

7. Exemplo de Teste em TypeScript

O arquivo de teste padrão geralmente localiza-se em tests/fowlgen_wars_contract.ts:

```
import * as anchor from "@coral-xyz/anchor";
import { Program } from "@coral-xyz/anchor";
import { FowlgenWarsContract } from "../target/types/fowlgen_wars_contract";

describe("fowlgen_wars_contract", () => {
  const provider = anchor.AnchorProvider.env();
  anchor.setProvider(provider);

  const program =
    anchor.workspace.FowlgenWarsContract as Program<FowlgenWarsContract>;
});
```

8. Configuração do Contrato para a Devnet

Esta etapa prepara o contrato para ser publicado na Solana Devnet.

Passo 8.1 — Atualizar o lib.rs

Edite o arquivo `programs/fowlgen_wars_contract/src/lib.rs` com o exemplo mínimo:

```
use anchor_lang::prelude::*;

declare_id!("1111111111111111111111111111111111111111111111111111111111111111");

#[program]
pub mod fowlgen_wars_contract {
    use super::*;

    pub fn initialize(ctx: Context<Initialize>) -> Result<()> {
        msg!("FOWLGEN WARS Online");
        msg!("Signer: {}", ctx.accounts.signer.key());
        Ok(())
    }
}

#[derive(Accounts)]
pub struct Initialize<'info> {
    #[account(mut)]
    pub signer: Signer<'info>,
}
```

Nota: O Program ID será sincronizado na etapa seguinte.

Passo 8.2 — Configurar o Anchor.toml

Na raiz do projeto, abra o arquivo `Anchor.toml` e configure a Devnet:

```
[toolchain]

[features]
seeds = true
skip-lint = false

[programs.devnet]
fowlgen_wars_contract = "SEU_PROGRAM_ID_AQUI"

[registry]
url = "https://api.apr.dev"

[provider]
cluster = "devnet"
wallet = "~/.config/solana/id.json"

[scripts]
test = "yarn run ts-mocha -p ./tsconfig.json -t 1000000 tests/**/*.ts"
```

9. Sincronização do Program ID

Passo 9.1 — Executar anchor keys sync

O Program ID precisa estar sincronizado entre `declare_id!()` no `lib.rs`, `Anchor.toml` e a chave localizada em `target/deploy/`. Execute:

```
anchor keys sync
anchor build
```

10. Compilação Final e Geração do IDL

Passo 10.1 — Compilar o Contrato

Execute o comando de compilação:

```
anchor build
```

O Anchor irá compilar o contrato Rust, gerar o programa `.so`, gerar o IDL e atualizar os arquivos de build. O IDL estará disponível em `target/idl/fowlgen_wars_contract.json`.

Sempre que a interface do contrato for alterada (novas instruções, contas ou parâmetros), execute novamente `anchor build` para gerar um IDL atualizado.

11. Exportar e Integrar o IDL ao Solana Unity SDK

O IDL (Interface Definition Language) descreve a interface do contrato para o cliente Unity, informando quais instruções existem, quais contas são necessárias, quais parâmetros podem ser enviados e como interagir com o programa.

Passo 11.1 — Localizar o IDL

Na raiz do projeto (`cd ~/fowlgen_wars_contract`), localize o arquivo em `target/idl/fowlgen_wars_contract.json`.

Passo 11.2 — Copiar o IDL para o Unity

Copie o arquivo para o seu projeto Unity, por exemplo:

```
UnityProject/
├── Assets/
│   └── Resources/
│       └── IDL/
│           └── fowlgen_wars_contract.json
```

Se o contrato for alterado posteriormente, execute novamente `anchor build` e substitua o IDL antigo no Unity.

Passo 11.3 — Validar a Integração com o Unity

Antes do deploy, confirme que: o contrato foi compilado sem erros, o Program ID está sincronizado, o IDL foi gerado e copiado para o Unity, e o Unity está utilizando o IDL correspondente à versão atual do contrato.

Importante: Nesta etapa o contrato ainda não precisa estar publicado na Devnet. O objetivo é preparar e validar os arquivos necessários para a integração com o Unity antes do deploy.

12. Publicar o Contrato na Solana Devnet

Passo 12.1 — Executar o Deploy

Com o contrato compilado, Program ID sincronizado, IDL preparado, carteira configurada e SOL de teste disponível, execute:

```
anchor deploy
```

O terminal deverá apresentar informações semelhantes a:

```
Deploying cluster: https://api.devnet.solana.com
Upgrade authority: /home/usuario/.config/solana/id.json
Deploying program "fowlgen_wars_contract"...
Program path: ../target/deploy/fowlgen_wars_contract.so
Program Id: <ENDEREÇO_DO_SEU_PROGRAMA>
Deploy success
```

O Program ID exibido no deploy deve corresponder exatamente ao Program ID configurado no projeto.

13. Validar o Programa na Devnet

Passo 13.1 — Executar os Testes na Devnet

Depois do deploy, execute os testes na rede Devnet:

```
anchor test --skip-local-validator
```

Esse comando permite executar os testes sem iniciar um novo validador local, utilizando a configuração de rede definida no projeto.

14. Verificar o Programa no Solana Explorer

Passo 14.1 — Localizar o Programa

Copie o Program ID exibido durante o deploy, abra o Solana Explorer, selecione a rede Devnet, cole o Program ID na barra de busca e verifique se o programa aparece como Executable. Essa etapa confirma visualmente que o programa foi publicado com sucesso na Solana Devnet.

15. Fluxo Completo do Ambiente

A sequência final do processo de configuração do ambiente é a seguinte:

1. Stack Tecnológica
2. Instalar Ambiente WSL
3. Configurar Solana Devnet
4. Inicializar Projeto Anchor
5. Verificar Estrutura
6. Executar Testes Locais
7. Configurar Testes TypeScript
8. Configurar Contrato para Devnet
9. Sincronizar Program ID
10. Compilar e Gerar IDL
11. Integrar IDL ao Unity
12. Deploy na Devnet
13. Testar na Devnet
14. Verificar no Solana Explorer